Gruber Login – Sprechskript

Zum Ablesen · Gesamtdauer ca. 8–10 Minuten · Person A ca. 4–5 Min., Person B ca. 4–5 Min. · Kursiv = Regieanweisung

TEIL 1 · PERSON A – Idee, Aufbau, Login-Modi

1. Begrüßung und Idee (ca. 1 Min.)

Guten Tag, wir stellen Ihnen unser Projekt „Gruber Login“ vor. Das ist eine Login-Anwendung, bei der man bei der Registrierung selbst wählt, **wie sicher** die Anmeldung sein soll.

Die Idee dahinter: Nicht jede Seite braucht die gleiche Sicherheit. Deshalb bieten wir drei Stufen an, vom einfachen Passwort bis zur Authenticator-App.

2. Technischer Aufbau (ca. 1,5 Min.)

Die Anwendung besteht aus drei Docker-Containern:

- **Datenbank:** MySQL speichert Benutzer, Sessions und Codes.
- **Web:** Das ist unser Backend mit FastAPI in Python. Es liefert die Seiten aus und stellt die Schnittstellen bereit.
- **Cloudflared:** Ein Tunnel, der die Seite per HTTPS aus dem Internet erreichbar macht, ohne dass wir Ports am Router öffnen müssen.

Alles startet mit einem einzigen Befehl: **docker compose up**. Das Backend besteht aus einer Datei, **app.py**, mit etwa 600 Zeilen.

3. Die drei Login-Modi (ca. 2 Min.)

Modus 1: Nur Passwort

E-Mail und Passwort, danach ist man direkt eingeloggt. Das ist der einfachste Modus.

Modus 2: E-Mail-Code

Nach dem Passwort bekommt man einen sechsstelligen Code per E-Mail. Er ist zehn Minuten gültig. Das ist eine Zwei-Faktor-Authentifizierung.

Modus 3: Authenticator-App

Bei der Registrierung wird ein QR-Code angezeigt, den man mit einer App wie Google Authenticator scannt. Danach braucht jeder Login den Code aus der App. Er ändert sich alle 30 Sekunden und funktioniert ohne Internet. Grundlage ist der Standard RFC 6238.

Übergabe: „Wie ein Login technisch abläuft und wie wir die Daten schützen, erklärt dir jetzt [Name Person B].“

TEIL 2 · PERSON B – Ablauf, Sicherheit, Demo, Fazit

4. Ablauf eines Logins (ca. 1,5 Min.)

1. Der Nutzer gibt E-Mail und Passwort ein. Der **Browser berechnet daraus einen SHA-256-Hash**. Das Klartext-Passwort wird nie über das Netz gesendet.
2. Der Server prüft diesen Hash mit **bcrypt** gegen die Datenbank.

3. Je nach Modus ist man jetzt eingeloggt, oder der Server verlangt den zweiten Faktor: E-Mail-Code oder Authenticator-Code.
4. Nach erfolgreicher Prüfung erzeugt der Server eine **Session**: einen zufälligen Token in einem HttpOnly-Cookie, sieben Tage gültig.
5. Die Willkommensseite prüft bei jedem Aufruf diesen Cookie. Ohne gültige Session wird man zurück zum Login geleitet.

5. Sicherheit (ca. 1,5 Min.)

- Passwörter liegen **nie im Klartext** in der Datenbank, sondern nur als bcrypt-Hash. Er lässt sich nicht zurückrechnen.
- Der Cookie ist **HttpOnly**, JavaScript kann ihn also nicht auslesen, und er wird standardmäßig **nur über HTTPS** gesendet.
- Wer sich mit einer schon vergebenen E-Mail erneut registriert, muss das **richtige Passwort** kennen. Fremde Konten lassen sich so nicht überschreiben.
- Beim Passwort-Reset verrät die Antwort nicht, ob eine E-Mail-Adresse existiert.
- Optional gibt es eine **Zeitsperre**: Login nur zu bestimmten Uhrzeiten, zum Beispiel 6 bis 22 Uhr.

Wie wird das Passwort gehasht? (zweistufig)

- **Stufe 1, im Browser:** SHA-256, einmal, ohne Salt und ohne Iterationen. Dafür sorgt das Klartext-Passwort, das den Browser nie verlässt.
- **Stufe 2, auf dem Server:** bcrypt mit **zufälligem Salt** pro Benutzer und einem **Kostenfaktor von 12**, also 4096 Runden. bcrypt ist

absichtlich langsam. Das Salt steckt im gespeicherten Hash.
- **Ehrliche Einschränkung:** Der SHA-256-Hash aus dem Browser ist ungesalzen und schnell. Wer ihn abfängt, kann sich damit anmelden. Deshalb brauchen wir trotzdem HTTPS. Den eigentlichen Schutz der gespeicherten Daten liefert bcrypt.
- **Verbesserung:** PBKDF2 im Browser mit der E-Mail als Salt und vielen Iterationen, oder das Passwort über HTTPS direkt an bcrypt senden.

Ehrlich gesagt: Das sind unsere Grenzen

Das Projekt ist ein Lern- und Demoprojekt, kein fertiges Produktivsystem. Uns sind folgende Punkte bewusst:

- Es gibt noch **keine Begrenzung der Fehlversuche** bei den Codes.
- Im Entwicklungsmodus stehen die Codes im Log. Das ist für die Demo praktisch, für Produktion aber nicht gedacht.

6. Live-Demo (ca. 2 Min.)

Vorher prüfen: docker compose läuft, Seite http://localhost:5000 oder die Tunnel-Adresse ist offen, Logs-Fenster (docker compose logs -f web) liegt bereit, Authenticator-App auf dem Handy ist bereit.

1. Registrieren im Modus **Passwort**: Man landet direkt auf der Willkommensseite.
2. Abmelden und neu registrieren im Modus **E-Mail-Code**: Den Code im Log-Fenster zeigen und eingeben.
3. Optional: Modus **Authenticator**: QR-Code scannen und den Code

aus der App eingeben.
4. Zeigen: **/welcome** ohne Login aufrufen. Man wird zum Login zurückgeleitet.

7. Fazit und Ausblick (ca. 0,5 Min.)

Wir haben gezeigt: eine Login-Anwendung mit drei Sicherheitsstufen, die Passwörter geschützt speichert, per Docker reproduzierbar startet und wenig Ressourcen braucht, etwa 0,5 bis 0,8 Gigabyte Arbeitsspeicher als Schätzwert.

Als nächster Schritt würden wir Fehlversuche begrenzen und automatisierte Tests ergänzen.

Vielen Dank! Haben Sie Fragen?

Mögliche Fragen und Kurzantworten

- **Warum SHA-256 im Browser und bcrypt auf dem Server?** Das Klartext-Passwort verlässt nie den Browser. bcrypt ist absichtlich langsam und schützt die gespeicherten Werte. Wichtig bleibt trotzdem HTTPS. Salt und Iterationen gibt es nur bei bcrypt (zufälliges Salt, Kostenfaktor 12 = 4096 Runden), nicht bei SHA-256.
- **Was ist der Unterschied zwischen E-Mail-Code und Authenticator?** Der E-Mail-Code hängt vom Postfach ab. Der Authenticator-Code wird auf dem Handy aus einem geheimen Schlüssel und der Uhrzeit berechnet.
- **Wozu der Cloudflare Tunnel?** Er liefert HTTPS und

Erreichbarkeit aus dem Internet, ohne Portfreigaben.

- **Was passiert, wenn man den Cookie stiehlt?** Dann hat man die Session. Deshalb HttpOnly und das Secure-Flag, das den Cookie nur über HTTPS sendet. Sessions laufen nach sieben Tagen ab.